Презентация по лабораторной работе 2

Иммитационное моделирование

Нджову Нелиа

Содержание I

Цель работы

Целью данной лабораторной работы является практическое изучение моделей SIR и Лотки-Вольтерры с использованием языка Джулии.

Задание

1. Изучить модель SIR с помощью julia

Задание

1. Изучить модель SIR с помощью julia
2. Изучить модель Лотки-Вольтерры с помощью julia

Теоретическое введение

Модель SIR есть классическая и фундаментальная математическая модель эпидемиологии, описывающая распространение инфекционного заболевания в закрытой популяции. Другими словами, модель SIR является одной из самых базовых моделей для описания временной динамики инфекционного заболевания в популяции

Теоретическое введение

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии:

— S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.

— I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.

— R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Теоретическое введение

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Она была независимо предложена в 1920-х годах:

— Альфредом Лоткой (1925) для химических реакций

— Витторио Вольтеррой (1926) для объяснения колебаний улова рыбы в Адриатическом море.

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

Выполнение лабораторной работы

Я копирую папку, в которую я установил необходимые пакеты из предыдущей лабораторной работы, в каталог lab2. Затем я создаю файл sir_ode.jl и копирую и вставляю код из руководства по лабораторной работе во вновь созданный файл(рис.1).

```
  GNU nano 7.2                    scripts/sir_ode.jl
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimmpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
```

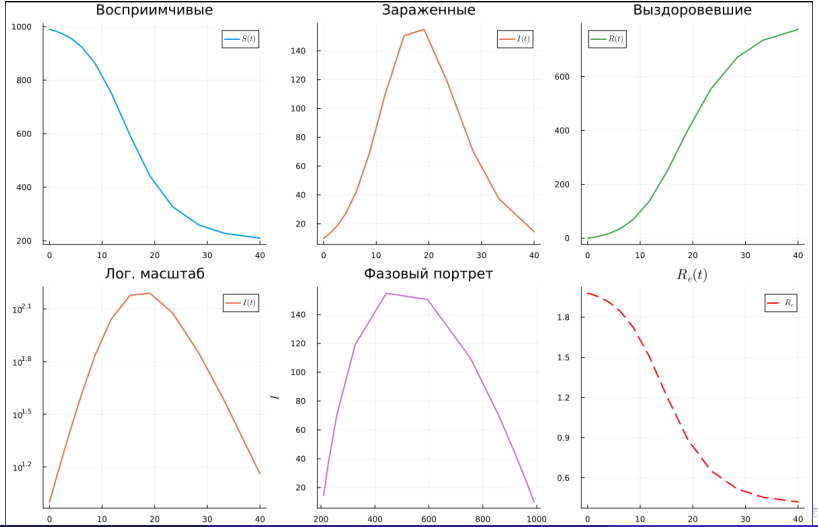
Выполнение лабораторной работы

Я преобразую код в литературный стиль(рис.2)

```
# # Модель SIR: Эпидемиологическое моделирование
#
# Этот скрипт реализует классическую модель SIR (Восприимчивые-Инфицированные-Выздоровевшие)
# для моделирования распространения инфекционного заболевания.
#
# **Версия с параметрическим сканированием**

# ## 1. Инициализация проекта и загрузка пакетов
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
using JLD2
using CSV

# Создание директорий для результатов
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## 2. Определение модели SIR
#
# Модель описывается системой дифференциальных уравнений:
#
# ```math
```

Выполнение лабораторной работы

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так(рис.3)



Выполнение лабораторной работы

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.4)

Нижний ряд

In [23]:
```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2,
    yscale=:log10, title="Лог. масштаб")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=2,
    title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, linewidth=2,
    title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```

Out[23]:

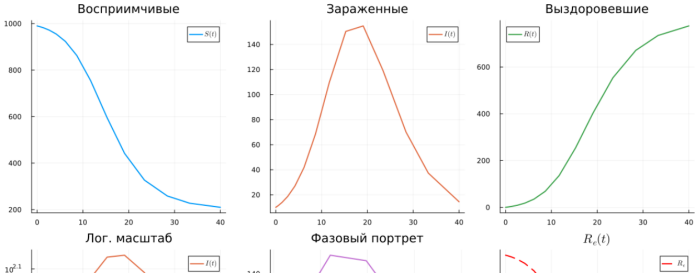


Рисунок 4: запуск кода из jupyter notebook

Выполнение лабораторной работы

Я добавляю в код расчет для набора параметров в литературном стиле, чтобы поэкспериментировать с ним.(рис.5)

```
Первые 10 строк:
10×9 DataFrame
 Row │ γ        final_recovered  u0                  R0        c         peak_time  peak_infected  β         tspan
     │ Float64  Float64          Array…              Float64   Float64   Float64    Float64        Float64   Tuple…
─────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────────
   1 │   0.2          33.1658    [990.0, 10.0, 0.0]  0.75       5.0       0.0        10.0           0.03      (0.0, 40.0)
   2 │   0.25         23.9815    [990.0, 10.0, 0.0]  0.6        5.0       0.0        10.0           0.03      (0.0, 40.0)
   3 │   0.33         18.1087    [990.0, 10.0, 0.0]  0.454545   5.0       0.0        10.0           0.03      (0.0, 40.0)
   4 │   0.5          14.212     [990.0, 10.0, 0.0]  0.3        5.0       0.0        10.0           0.03      (0.0, 40.0)
   5 │   0.2         363.541     [990.0, 10.0, 0.0]  1.5       10.0      32.6        69.7251        0.03      (0.0, 40.0)
   6 │   0.25        187.764     [990.0, 10.0, 0.0]  1.2       10.0      32.2        23.1073        0.03      (0.0, 40.0)
   7 │   0.33         65.5797    [990.0, 10.0, 0.0]  0.909091  10.0       0.0        10.0           0.03      (0.0, 40.0)
   8 │   0.5          24.3649    [990.0, 10.0, 0.0]  0.6       10.0       0.0        10.0           0.03      (0.0, 40.0)
   9 │   0.2         813.099     [990.0, 10.0, 0.0]  2.25      15.0      18.8       199.614         0.03      (0.0, 40.0)
  10 │   0.25        689.232     [990.0, 10.0, 0.0]  1.8       15.0      20.2       123.484         0.03      (0.0, 40.0)

============================================================
АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ
============================================================
```

Рисунок 5: добавление кода

Выполнение лабораторной работы

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так.В данных видно что при $R_0 < 1$ количество зараженных не растет (остается 10 человек) а при $R_0 > 1$ начинается вспышка, чем больше контактов (c), тем сильнее эпидемия и чем быстрее выздоровление (γ), тем слабее эпидемия.(рис.6)

10×12 DataFrame

Row	α	u0	б	β	mean_prey	tspan	prey_amplitude	γ	mean_predator	x_star	y_star	preda tor_amplitude
	Float64	Array…	Float64	Float64	Float64	Tuple…	Float64	Float64	Float64	Float64	Float64	Float 64
1	0.05	[40.0, 9.0]	0.005	0.01	39.7947	(0.0, 200.0)	13.0597	0.2	5.14151	40.0	5.0	3.29597
2	0.05	[40.0, 9.0]	0.005	0.01	59.0119	(0.0, 200.0)	28.0659	0.3	4.60024	60.0	5.0	5.9021
3	0.05	[40.0, 9.0]	0.005	0.01	80.7191	(0.0, 200.0)	53.5718	0.4	4.60874	80.0	5.0	10.2568
4	0.05	[40.0, 9.0]	0.005	0.01	100.421	(0.0, 200.0)	83.8135	0.5	4.2463	100.0	5.0	15.4675
5	0.05	[40.0, 9.0]	0.01	0.01	18.6958	(0.0, 200.0)	17.3045	0.2	5.21788	20.0	5.0	9.17241
6	0.05	[40.0, 9.0]	0.01	0.01	29.2117	(0.0, 200.0)	12.1603	0.3	5.06618	30.0	5.0	5.08093
7	0.05	[40.0, 9.0]	0.01	0.01	39.392	(0.0, 200.0)	9.2436	0.4	5.06534	40.0	5.0	3.30281
8	0.05	[40.0, 9.0]	0.01	0.01	50.0495	(0.0, 200.0)	14.9469	0.5	4.83414	50.0	5.0	4.82047
9	0.05	[40.0, 9.0]	0.015	0.01	13.0688	(0.0, 200.0)	19.401	0.2	6.43321	13.3333	5.0	16.8145
10	0.05	[40.0, 9.0]	0.015	0.01	19.9553	(0.0, 200.0)	16.8798	0.3	5.8233	20.0	5.0	11.2713

Рисунок 6: запуск кода

Выполнение лабораторной работы

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.7)

```
In [26]: param_grid = Dict(
             :α => [0.05, 0.1, 0.15, 0.2],          # скорость размножения жертв
             :β => [0.01, 0.02, 0.03, 0.04],         # скорость поедания жертв
             :δ => [0.005, 0.01, 0.015, 0.02],       # коэффициент конверсии
             :γ => [0.2, 0.3, 0.4, 0.5],             # смертность хищников
             :u0 => [[40.0, 9.0]],                    # начальные условия (фиксированы)
             :tspan => [(0.0, 200.0)]                 # время моделирования (фиксировано)
         )

Out[26]: Dict{Symbol, Vector} with 6 entries:
           :α     => [0.05, 0.1, 0.15, 0.2]
           :γ     => [0.2, 0.3, 0.4, 0.5]
           :u0    => [[40.0, 9.0]]
           :δ     => [0.005, 0.01, 0.015, 0.02]
           :β     => [0.01, 0.02, 0.03, 0.04]
           :tspan => [(0.0, 200.0)]
```

Рисунок 7: запуск кода из jupyter notebook

Выполнение лабораторной работы

Я создаю файл lv_ode.jl и копирую и вставляю код из руководства по лабораторной работе во вновь созданный файл(рис.8).

```
  GNU nano 7.2                    scripts/lv_ode.jl *
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
# Дополнительный анализ: чувствительность к параметрам
println("\n\n" * "="^60)
```

Рисунок 8: Код из лабораторного руководства

Выполнение лабораторной работы

Я преобразую код в литературный стиль(рис.9)

```
# # Модель Лотки-Вольтерры (Хищник-Жертва)
#
# Этот скрипт реализует классическую модель Лотки-Вольтерры для оп
# динамики взаимодействия двух видов: хищников и жертв.

# ## 1. Инициализация проекта и загрузка пакетов

using DrWatson
@quickactivate "project"

using DifferentialEquations  # Решение ОДУ
using DataFrames             # Работа с данными
using Plots                   # Визуализация
using LaTeXStrings            # Красивые формулы
using Statistics              # Статистический анализ
using FFTW                    # Спектральный анализ

# Создание директорий для результатов
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## 2. Математическое описание модели
#
# Модель Лотки-Вольтерры описывается системой дифференциальных ура
```

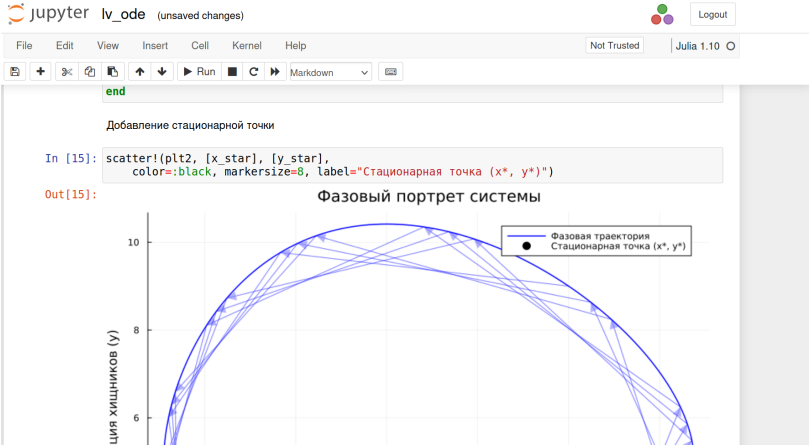
Выполнение лабораторной работы

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так(рис.10)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2
026-1--study--simulationmod/labs/lab02/project$ julia --project=. s
cripts/lv_ode.jl
┌ Warning: Verbosity toggle: dense_output_saveat
│   Dense output is incompatible with saveat. Please use the SavingC
allback from the Callback Library to mix the two behaviors.
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/MfUJb/s
rc/solve.jl:133
============================================================
МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА)
============================================================

Параметры модели:
  α (скорость размножения жертв) = 0.1
  β (скорость поедания жертв) = 0.02
  δ (коэффициент конверсии) = 0.01
  γ (смертность хищников) = 0.3

Начальные условия:
  Жертвы (x₀) = 40.0
  Хищники (y₀) = 9.0

Стационарные точки (положения равновесия):
```

Выполнение лабораторной работы

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.11)



Выполнение лабораторной работы

Я добавляю в код расчет для набора параметров в литературном стиле, чтобы поэкспериментировать с ним(рис.12)

```
# - **γ**: смертность хищников

param_grid = Dict(
    :α => [0.05, 0.1, 0.15, 0.2],        # ск
    :β => [0.01, 0.02, 0.03, 0.04],       # с
    :δ => [0.005, 0.01, 0.015, 0.02],     # к
    :γ => [0.2, 0.3, 0.4, 0.5],           # с
    :u0 => [[40.0, 9.0]],                  #
    :tspan => [(0.0, 200.0)]               #
)

# Создаём все возможные комбинации параметров
```

Рисунок 12: добавление кода

Выполнение лабораторной работы

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так. Наблюдения: Когда γ (смертность хищников) высокая, равновесная численность жертв (x) *увеличивается. Когда δ (коэффициент конверсии) низкий, колебания становятся больше. Когда α (рождаемость жертв) низкая, система все еще может иметь большие колебания, если другие параметры позволяют. Когда β (поедание жертв) высокий, равновесная численность хищников (y)* уменьшается.

Выполнение лабораторной работы

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок. (рис.13)

- γ: смертность хищников

In [ ]:
```
param_grid = Dict(
    :α => [0.05, 0.1, 0.15, 0.2],        # скорость размножения жертв
    :β => [0.01, 0.02, 0.03, 0.04],       # скорость поедания жертв
    :δ => [0.005, 0.01, 0.015, 0.02],     # коэффициент конверсии
    :γ => [0.2, 0.3, 0.4, 0.5],           # смертность хищников
    :u0 => [[40.0, 9.0]],                  # начальные условия (фиксированы)
    :tspan => [(0.0, 200.0)]               # время моделирования (фиксировано)
)
```

Создаём все возможные комбинации параметров вручную Всего комбинаций: 4 × 4 × 4 × 4 = 256 экспериментов

In [ ]:
```
println("\nСоздание комбинаций параметров...")

all_params = []
for α in param_grid[:α]
    for β in param_grid[:β]
        for δ in param_grid[:δ]
            for γ in param_grid[:γ]
```

Рисунок 13: запуск кода из jupyter notebook

Выводы

Выполнив эту лабораторную работу, я получила практическое знание о моделях SIR и Лотки-Вольтерры с использованием языка Джулии.